recap:

① 线性分类器

形式化输入 × 权重矩阵 = scores.
张量 weight martix

关于线性分类的3种解释：

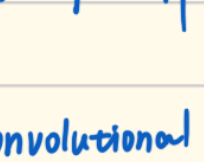
(1) 矩阵.权重 (2) 视觉 (3) 超维空间

② 评价模型的好坏：损失函数.(Loss function)

Softmax: $L_i = -\log\left(\frac{e^{s_{yi}}}{\sum_j e^{s_j}}\right)$

SVM:(铰链损失) $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{yi} + 1)$.

③ 得到更好的参数：优化(Optimization)



目的：在参数空间中找到最低Loss点

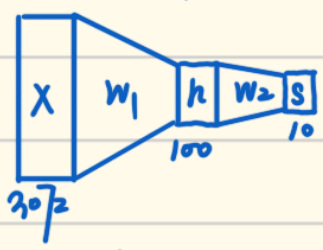
(1) SGD：随机梯度下降 (3) Adam.(★)

(2) RMSprop

④ powerful model

线性模型的缺陷：{视觉：单个类别评分向量需总结所有知识
几何：超平面切分区域，无法适用非可切分特征.

多层权重矩阵 + 非线性连接.



代价是面的添加不多，但其能力更强大了。

⑤ 多层矩阵 + 线性层的优化：

计算图：前向传播计算Loss. 后向传播计算gradients

↳ "反向传播"：global problem → Local problem

$\left(\frac{\partial L}{\partial W}\right)$ $\left(\frac{\partial Out}{\partial W} \cdot \frac{\partial L}{\partial Out}\right)$

神经网络：

① build：全连接，CNN，Relu等激活层，池化层.

② train：正/反向传播 → Loss与梯度

Optimization → 梯度 - 权重更新

③ test：训练.测试.验证集.

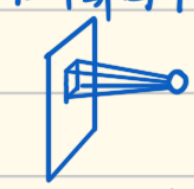
{超参数选择

Perceiving and Understanding the Visual World.

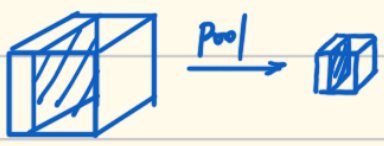
感知和理解视觉世界

Convolutional Networks 卷积神经网络.

① 在计算图中意义："新"节点(过去节点：全连接层(矩阵乘法)，reLU等非线性层，Loss function...)



Convolution Layer



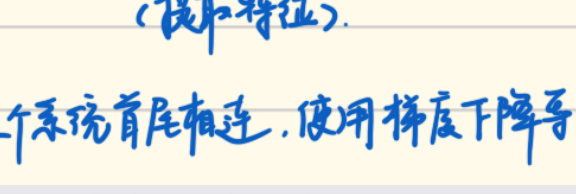
Pooling Layer

② Pixel Space or Image features.

(对神经网络的输入：图像 → 输入)

- Pixel：图像 → 像素向量/张量（每个读为RGB值）→ 输入
- Feature：图像 —func→ 提取出特征 → 输入.

example：
- Color Histogram. 颜色直方图 → 颜色信息
- Histogram of Oriented Gradients (HoG). 定向梯度图 → 结构信息.

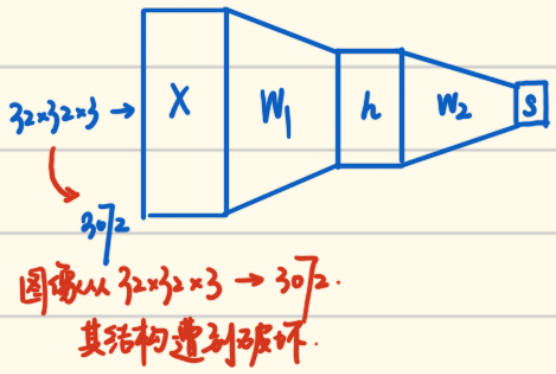


与前面提到的区别：

其实与直接 pixel 输入相比结构几乎一致，只区别在人为定义了…，梯度下降学习了…

"梯度下降也许比你程序员的直觉更优秀".
↳ 难以写下普适/最优的特征提取器.
} 数据驱动. flexible.

③ Problem on Nearal Networks.



图像从 32×32×3 → 3072.
其结构遭到破坏.

考虑将什么放入网络.

使其能更"尊重"二维结构？

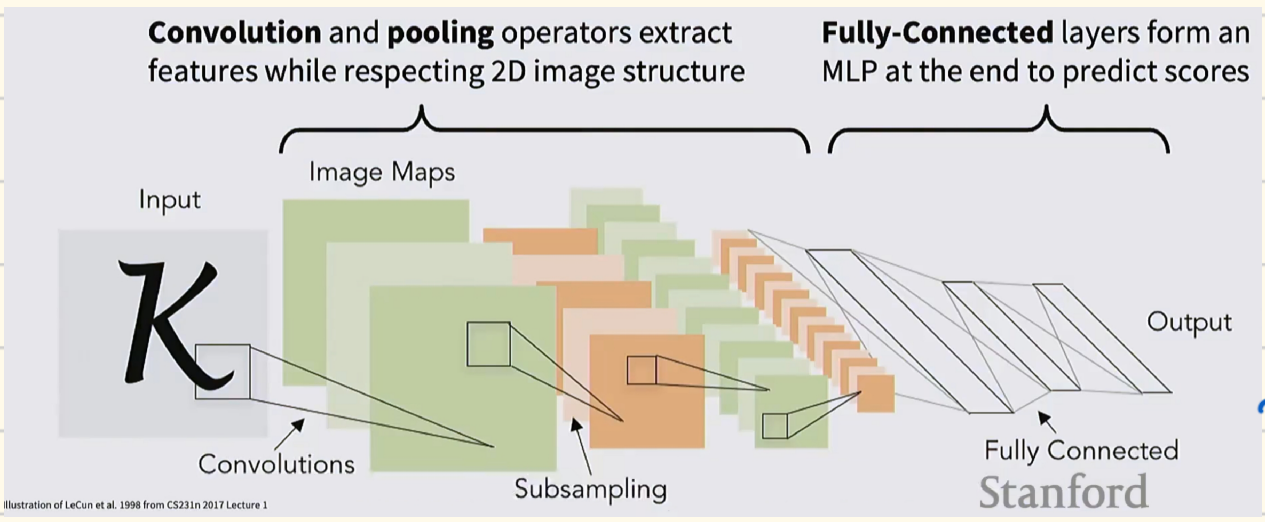
↳ 卷积层.

相比全连接层：{局部性 / 平移等变性} 加入"先验知识"使其更易收敛.

④ Convolutional Neural Network. 卷积神经网络.

input → Convolution/Pooling operators. → Fully-Connected layers → Score.
(提取特征). (MLP)

整个系统首尾相连，使用梯度下降等训练.



来自1998年论文。效果相当不错。

然而当时受限于计算能力，其效果呈现非常昂贵，直到2010s左右。

~2012-2020. 卷积网络在这个时代具有强统治力，解决几乎所有问题.

① 图像分类，检测
② 图像 → 文字，摘要问题
③ (最近) 自然语言 → 图像生成.

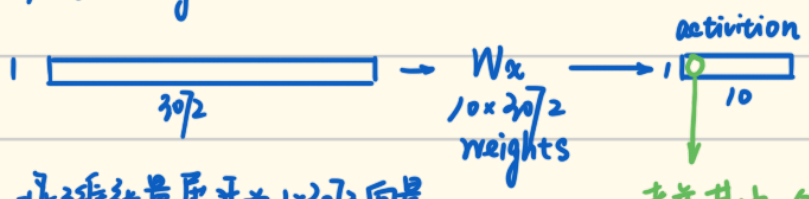
自2015开课至今10年，领域依然在扩展。

卷积网络 —2020→ transformers. 将CNN用transformer取替.
(2017. 文本 → 2021. 所有图像架构). 更好表现

Today: Convolutional Networks.

recap: Fully Connected Layer.

32×32×3 image



将3维张量展开为1×3072向量.

考虑其中一个点. 它来自于 (图像) 1×3072 × (权重) 3072×1

实质上是两向量点乘

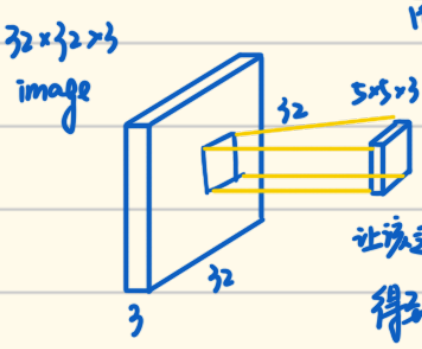
权重 → 图像

当方向一致时取得最大值.

本质上是模板匹配

基于"匹配"的观点: "卷积层"匹配不再是等大的展开图像

而是仅匹配输入的一小部分.



一张特小图像(滤波器).

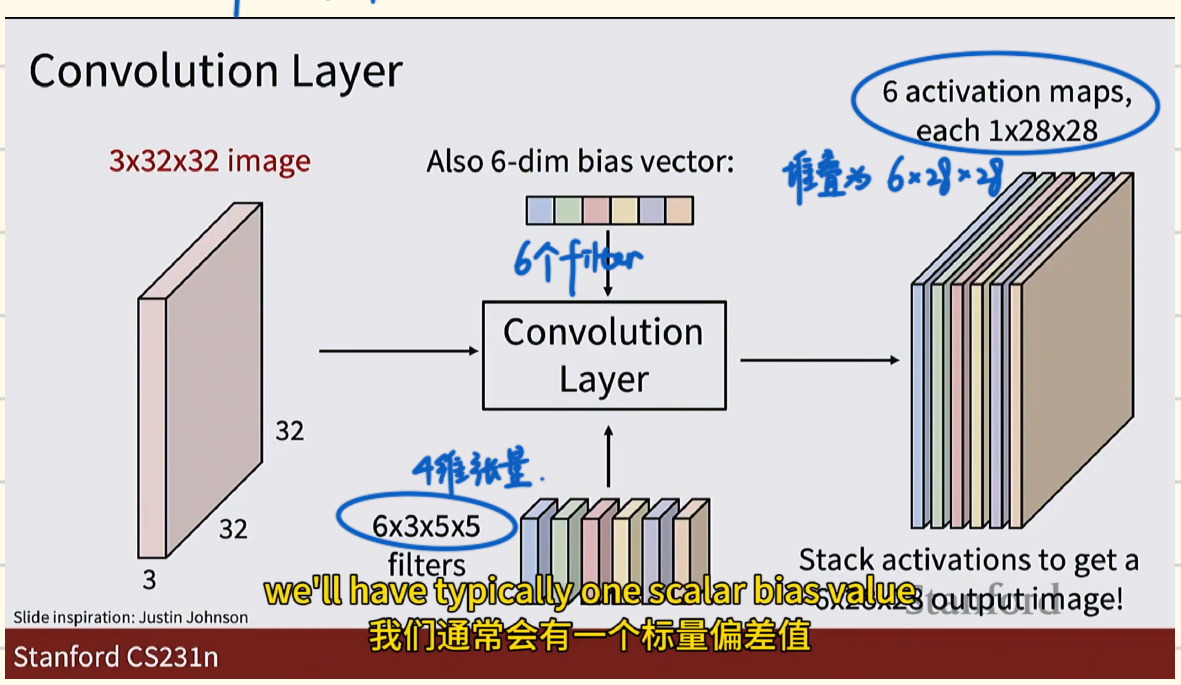
让该滤波器在image上滑动.

得到一系列分数.

(例如32×32×3来匹配5×5×3. 应得28×28×1的分类表).

增加filter. filter1. filter2. filter3 ...

最终得到多个结果



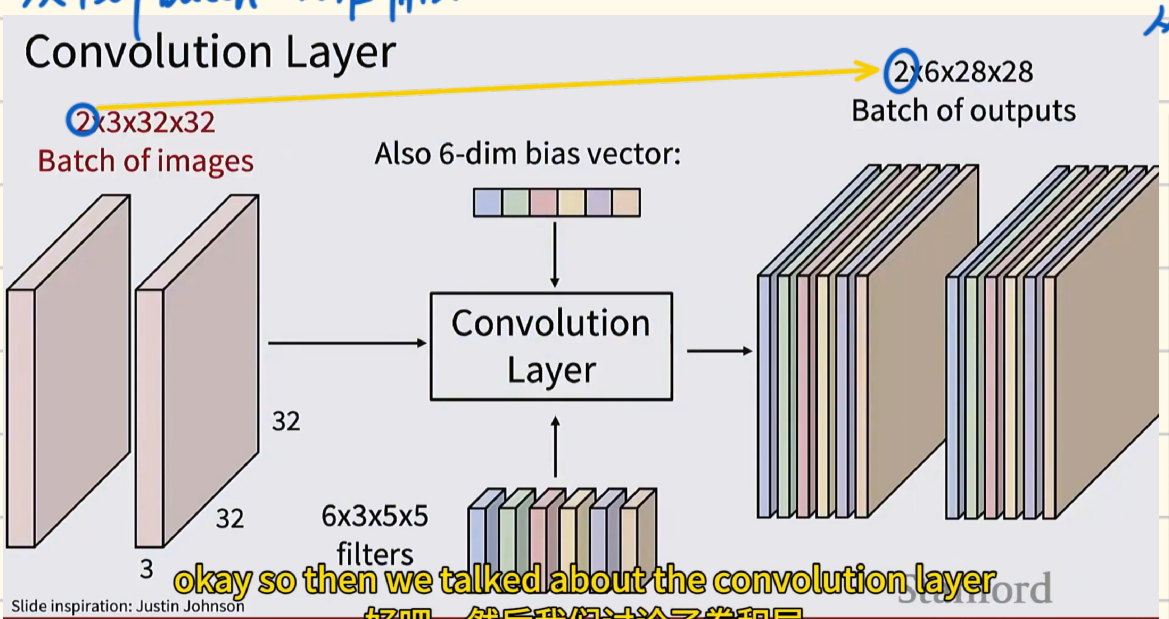
"bias": 对每个filter的乘积都存在1个bias标量

所以共得6维bias. (1×6).

filter并非先被定义的, 相反. 它通过学习得到.

{"6个filter" / "size of filter"} → hyperparameters.

从1到batch: 四维输入.



从一张图到n张图, 每个输出都是3维张量.

$N \times C_{in} \times H \times W$

↓ 卷积层. $C_{out} \times C_{in} \times k_w \times k_h$ (filters) + bias

C_{out}个过滤器, 每个有$k_w \times k_h$形状

$N \times C_{out} \times H' \times W'$

N个3维张量, 每个3维张量由C_{out}个$H' \times W'$的分类矩阵形成(特征平面).

C_{in}: 这里是RGB通道. 3

C_{out}: 过滤器的数量.

问题：卷积是线性的.

○ —Conv→ ○ —Conv→ ○ …

多卷积层堆叠，实际只起到单层效果.

↳加入激活函数.

○ —Conv/ReLU→ ○ —Conv/ReLU→ ○

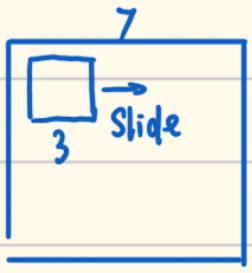
"卷积核(filter)的可视化"：

特征元素(第一层)：{ 点：颜色对比
重直、水平、对角边缘.

↓

高层：低层过滤器的组合.
成为复杂形状.

"如何'分化'filter"：随机初始化，打破对称性.



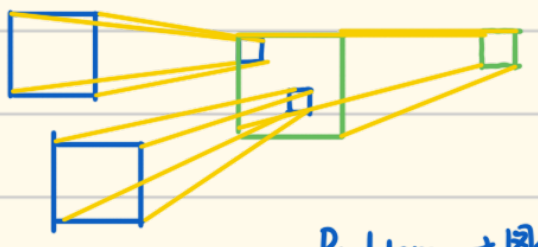
Input：7×7
filter：3×3
Output：5×5.

Input W，filter：k.
Output：W−k+1.

怎么让卷积层前后的"大小不变"呢？

填充0：填尺寸. → 边缘问题：其实实践中没什么区别

"感受野"，Receptive Fields.

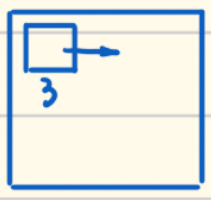


RF：输出图上的每个点对应原始图像上区域的大小.
越高层的条目，对底层条目依赖的数量就越多。

$RF_{i-1} = (RF_i - 1) \times S_i + k_i$

Problem：大图像若要让卷积包含整个图，或许需要很多层.（感受野线性增长）.

"Strided"：步幅、跳过.



滑动的时候一次走多步.

7×7，3×3 → Output：3×3

Input：W. Filter：K
padding：P. Stride：S
（填充）

Output：$(W-K+2P)/S+1$

此时若每层都有 Stride. 感受野就在每层指数增长.

"Translation Equivariance."：平移不变性.